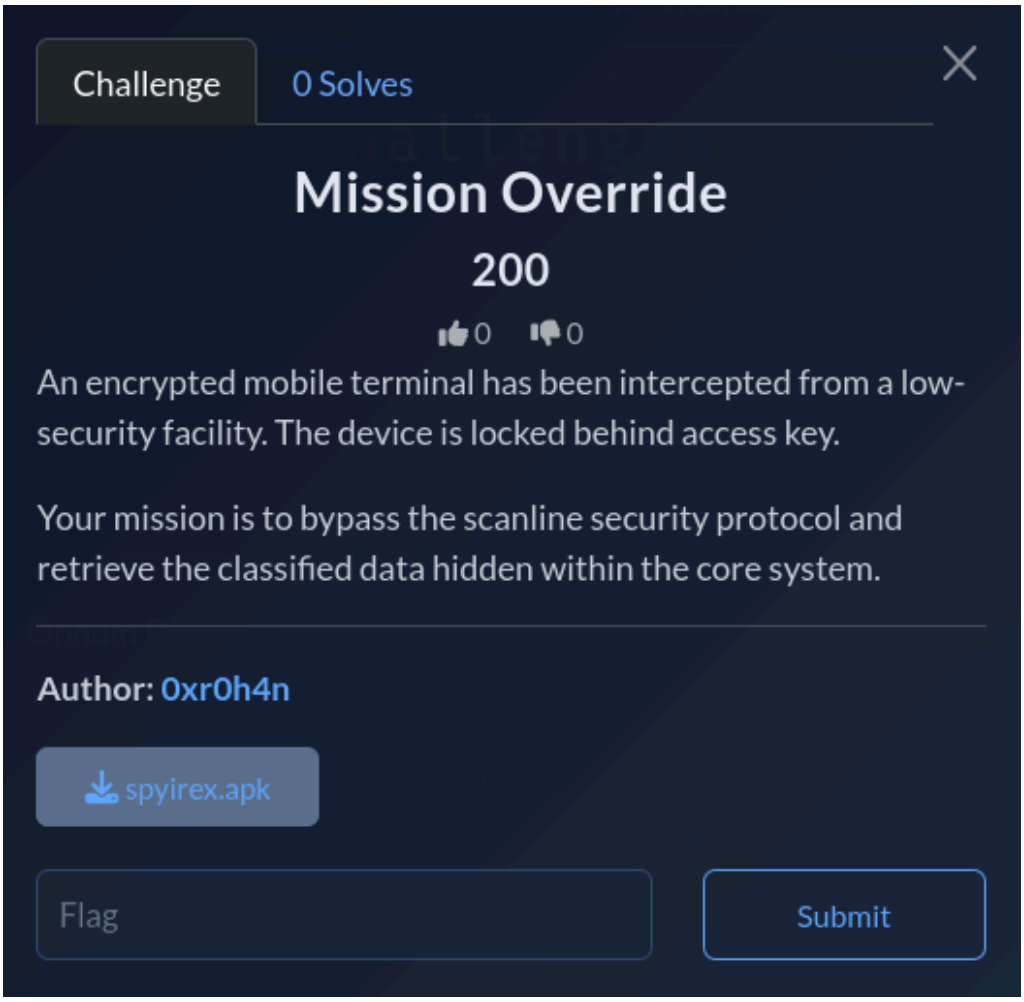
On 24th January, 2026. Jerusalem college of Engineering, Department of Cyber Security and Cyber Nexus Organizes **FLAG OVER FLOW.** This event is an Inter-college CTF event!

Challenge Authors: 0xr0h4n, Subadevan C, orochi, N1rm4l_D4rkn3ss18, d3xt3r

Challenge Categories: Cryptography, Web, Reverse Engineering, Forensic, OSINT, Stegnograpy

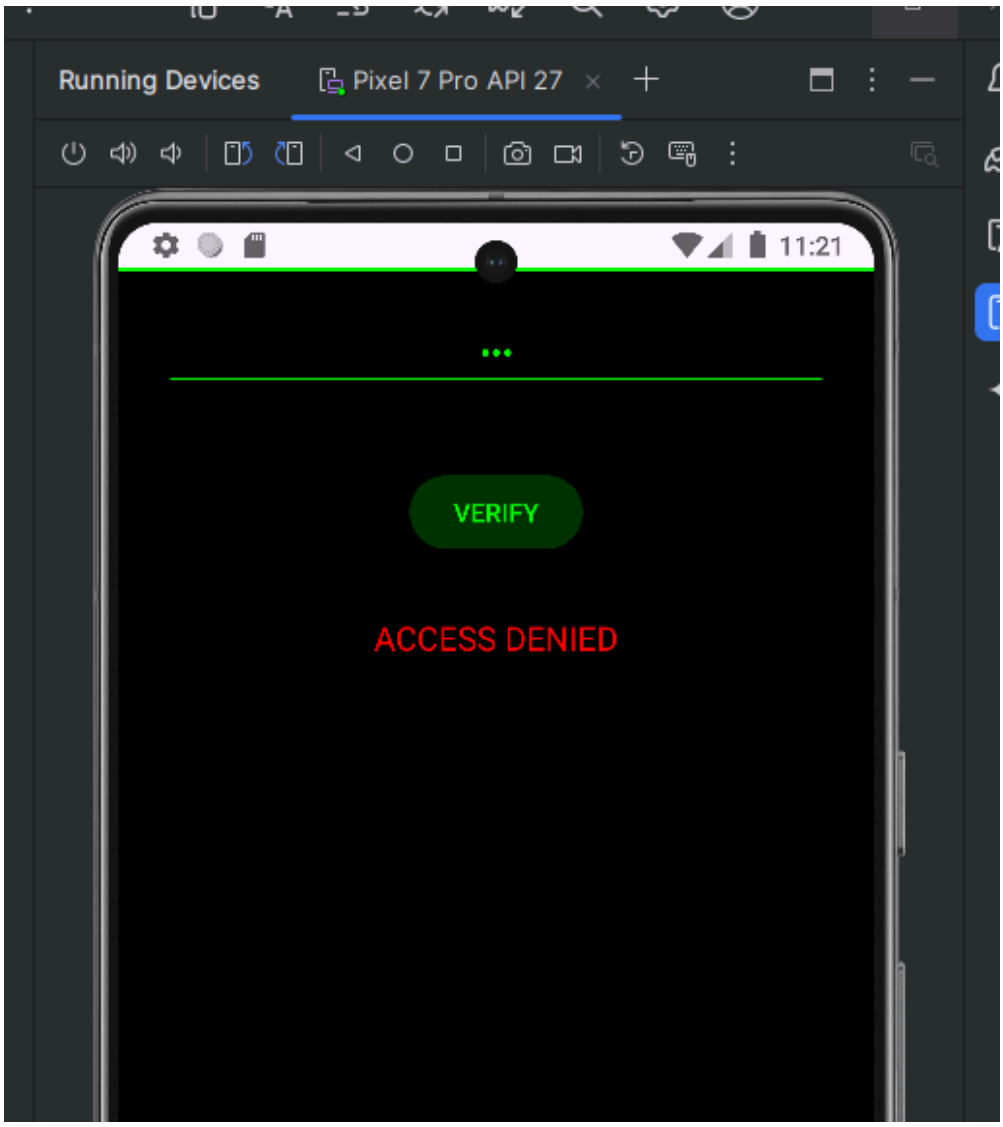
Reverse Engineering:

Mission Override



File: spyirex.apk

In this challenge, you have to enter the right password inorder to display the flag, so lets decompile our apk using jadx-gui



Global search (Ctrl+Shift+F), find for “MainActivity” where suspicious activities hide.

Press enter or click to view image in full size

```
23          this.passwordInput = (EditText) findViewById(R.id.passwordInput);
24          this.checkBtn = (Button) findViewById(R.id.checkBtn);
25          this.resultText = (TextView) findViewById(R.id.resultText);
26          this.flagImage = (ImageView) findViewById(R.id.flagImage);
27          this.scanLine = findViewById(R.id.scanLine);
28          this.checkBtn.setOnClickListener(new View.OnClickListener() { // from class: com.ctf.spyirex.MainActivit
29              @Override // android.view.View.OnClickListener
30              public final void onClick(View view) {
31                  this.f$0.m158lambda$onCreate$0$comctfspyirexMainActivity(view);
32              }
33          });
34      }
35
36      /* renamed from: lambda$onCreate$0$com-ctf-spyirex-MainActivity, reason: not valid java name */
37      /* synthetic */ void m158lambda$onCreate$0$comctfspyirexMainActivity(View v) {
38          String input = this.passwordInput.getText().toString().trim();
39          if (verify(input)) {
40              this.resultText.setVisibility(8);
41              this.flagImage.setVisibility(0);
42          } else {
43              this.flagImage.setVisibility(8);
44              this.resultText.setVisibility(0);
45              this.resultText.setText("ACCESS DENIED");
46          }
47      }
48
49      private boolean verify(String s) {
50          int[] enc = {99, 114, 96, 96, 100, 124, 97, 119, 122, 96, 99, 114, 96, 96};
51          if (s.length() != enc.length) {
52              return false;
53          }
54          for (int i = 0; i < enc.length; i++) {
55              if ((s.charAt(i) ^ 19) != enc[i]) {
56                  return false;
57              }
58          }
59          return true;
60      }
61  }
```

Here your password is hiddien! So, this is the vulnerable one.w

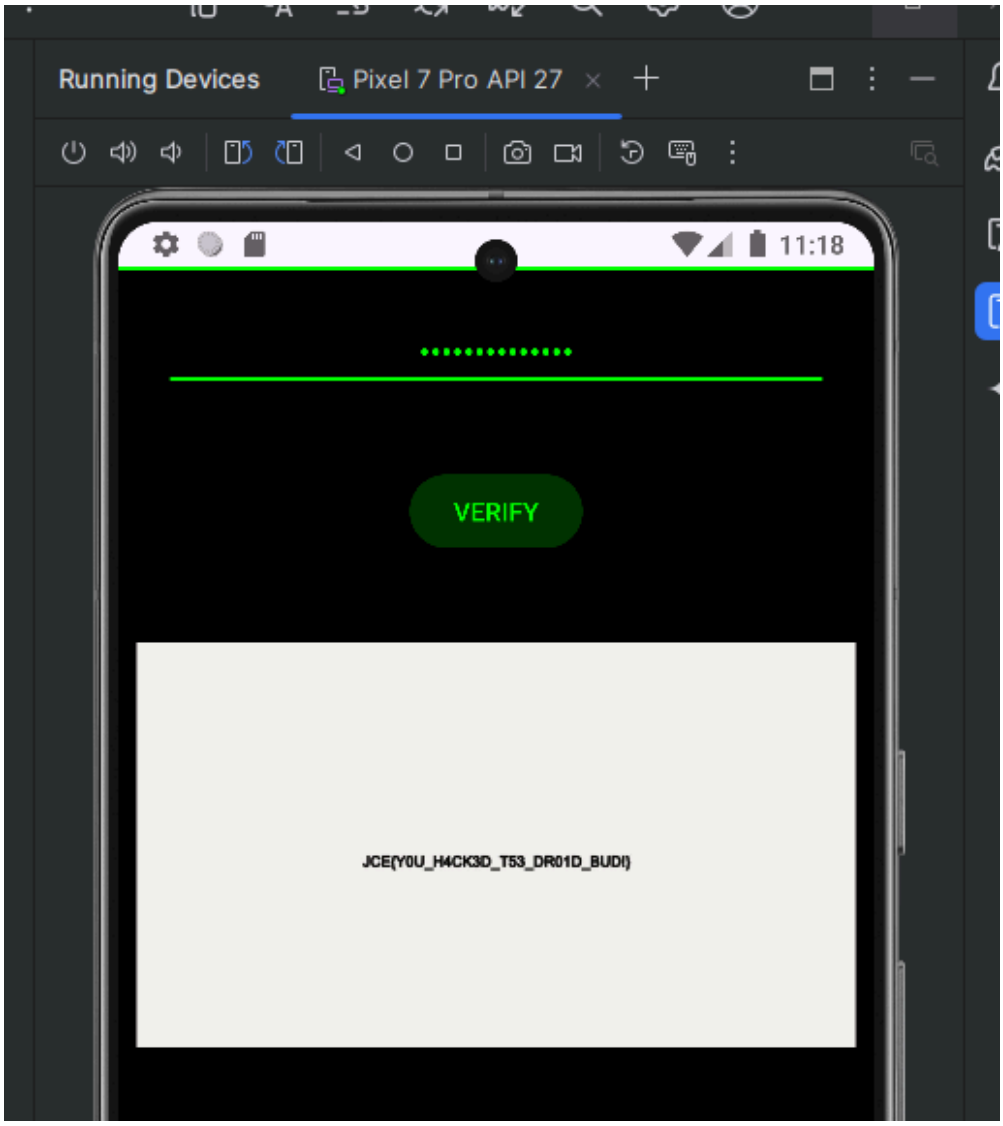
```
private boolean verify(String s) {
        int[] enc = {99, 114, 96, 96, 100, 124, 97, 119, 122,
96, 99, 114, 96, 96};
        if (s.length() != enc.length) {
            return false;
        }
        for (int i = 0; i < enc.length; i++) {
            if ((s.charAt(i) ^ 19) != enc[i]) {
                return false;
            }
        }
        return true;
    }
```

Lets crack our password!

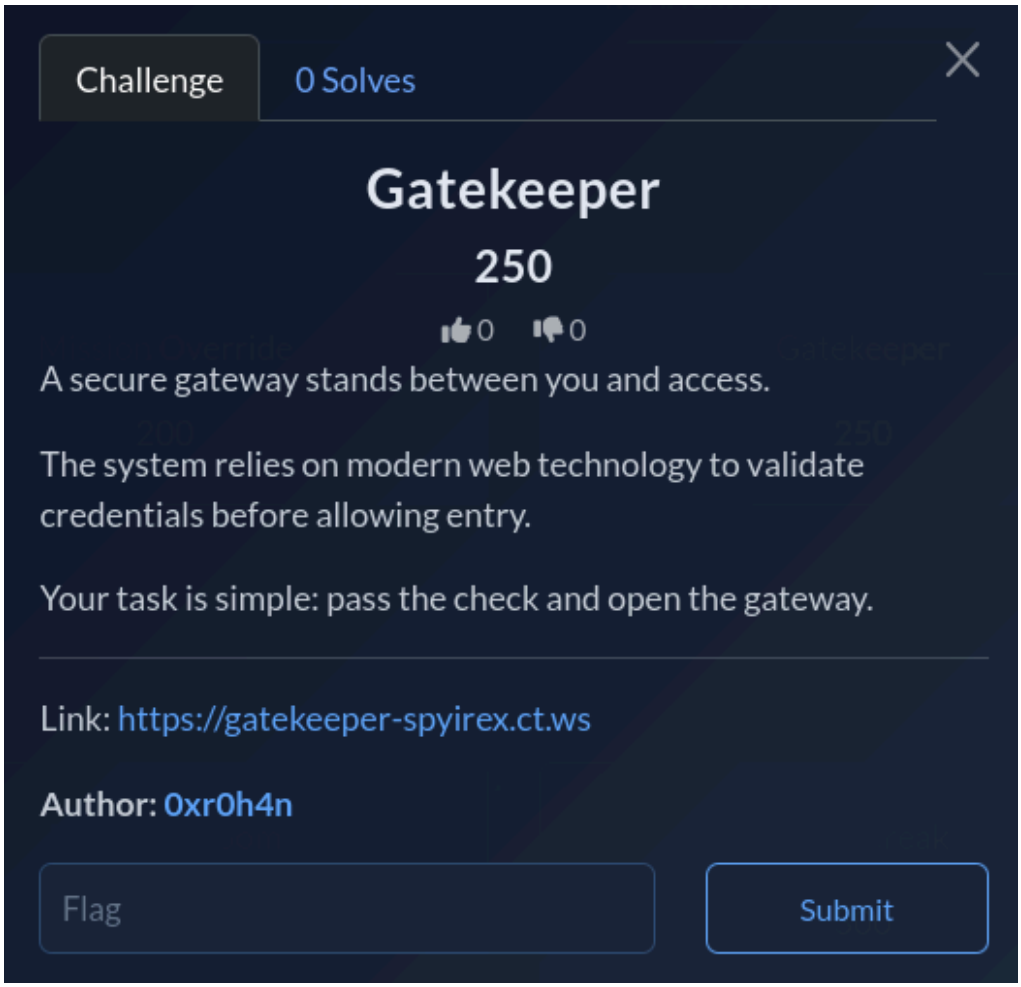
```
enc = [99, 114, 96, 96, 100, 124, 97, 119, 122, 96, 99, 114,
96, 96]
```

```
key = 19
password = ''.join(chr(e ^ key) for e in enc)
print(password)
```

The password is **passwordispass**. Let's display our flag!!



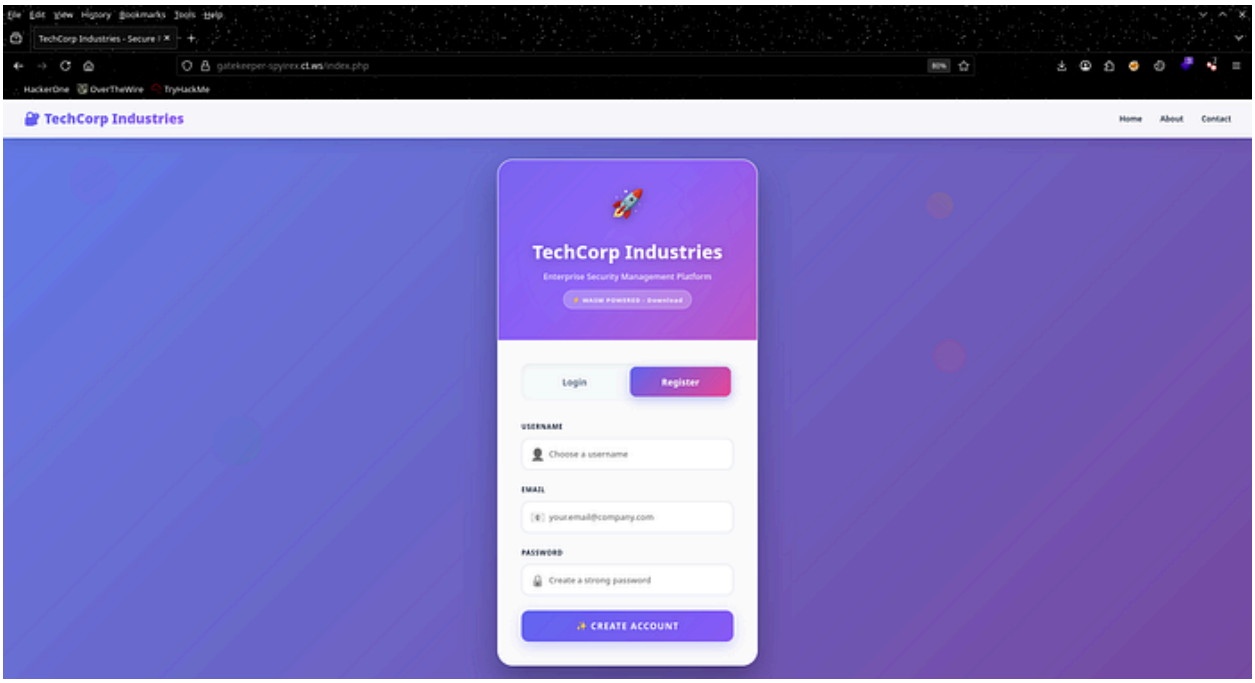

```
Flag: JCE{Y0U_H4CK3D_T53_DR01D_BUD!}
```



Site: https://gatekeeper-spyirex.ct.ws/

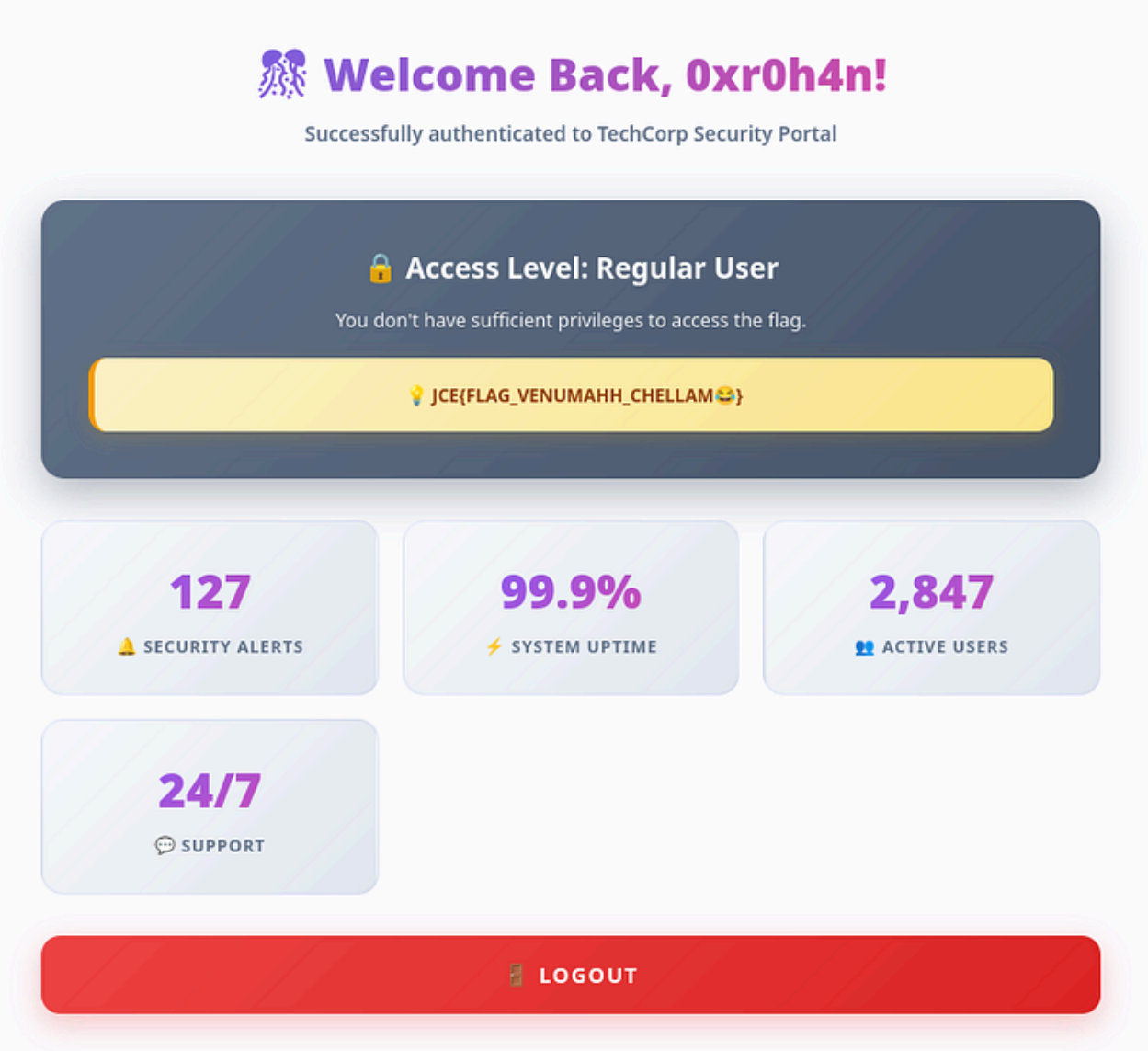
So, its a medium Web ASWM challenge. Where you need to login as admin with the right password to get the flag!

Press enter or click to view image in full size



This is the site. Now, lets register and login as a user. Test the security, does the flag reveal for normal users..

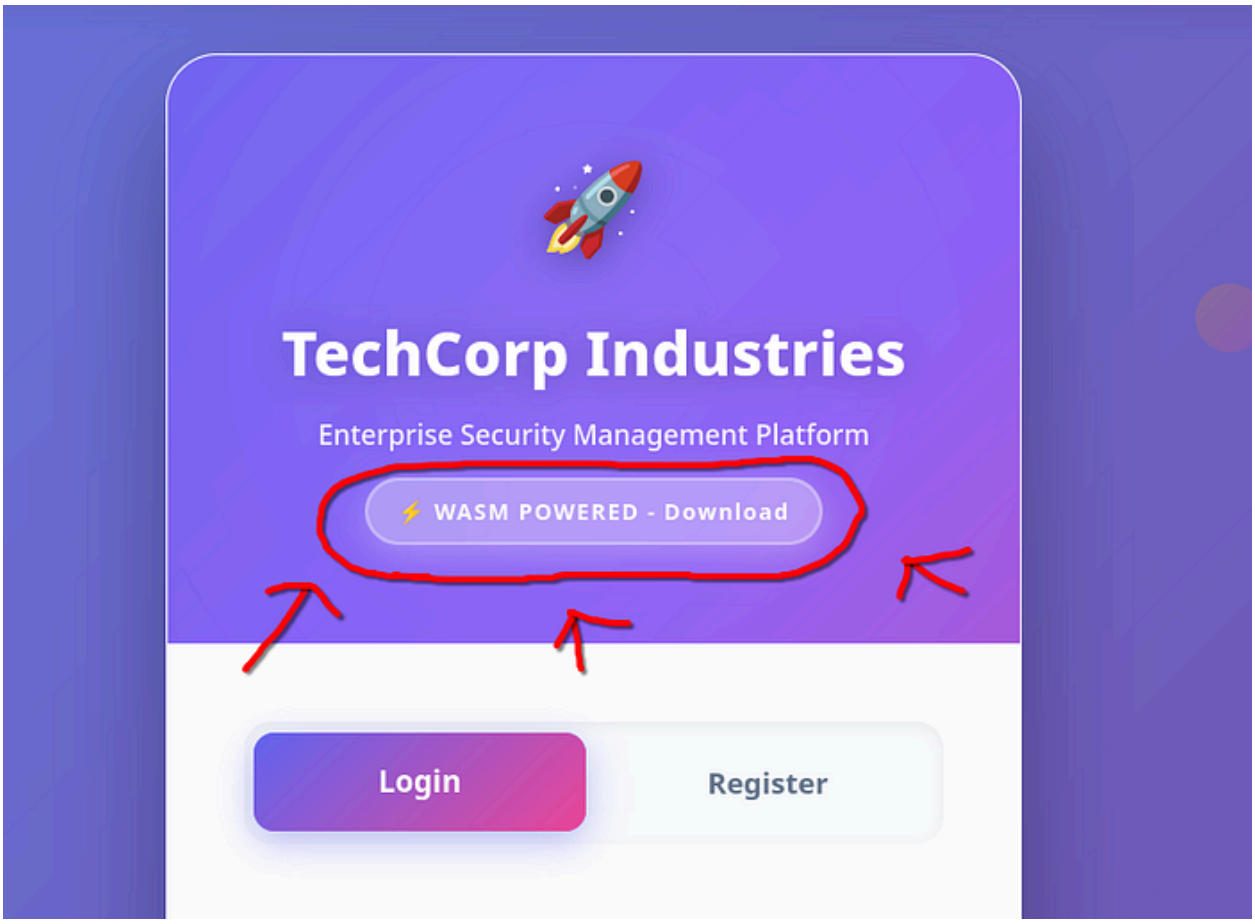
Press enter or click to view image in full size



It gives us fake flag ;(

But if you carefully looked in the index page, there is a button to download a WASM file(auth.wasm)

Press enter or click to view image in full size



Download it. and now lets decomplie it using wasm2wat

```
wasm2wat auth.wasm -o auth.wat
```

```
(module
  (type (;0;) (func (param i32 i32) (result i32)))
  (func (;0;) (type 0) (param i32 i32) (result i32)
    (local i32 i32 i32 i32)
    local.get 1
    i32.const 24
    i32.ne
    if  ;; label = @1
      i32.const 0
      return
    end
```

```
    i32.const 0
    local.set 2
    block  ;; label = @1
      loop  ;; label = @2
        i32.const 66
        local.get 2
        i32.const 3
        i32.mul
        i32.add
        local.set 3
        local.get 0
        local.get 2
        i32.add
        i32.load8_u
        local.set 4
        local.get 2
        i32.load8_u
        local.set 5
        local.get 4
        local.get 3
        i32.xor
        local.get 5
        i32.ne
        if  ;; label = @3
          br 2 (;@1;)
        end
        local.get 2
        i32.const 1
        i32.add
        local.set 2
        local.get 2
        local.get 1
        i32.lt_u
        br_if 0 (;@2;)
      end
    end
```

```
    i32.const 1)
  (memory (;0;) 1)
  (export "memory" (memory 0))
  (export "check" (func 0))
  (data (;0;) (i32.const 0) "\\15\\04\\1b\\06\\11\\18\\07\\08
\\1c\\122<(,>++*(,0\\c4\\d6\\d4"))
```

Now, the interesting and careful part → understand the code properly!!!!!!!!!

1)Identify the password check function (Becoz that's the validator)

```
(export "check" (func 0))
```

2)Notice the length check (Which is 24)

```
local.get 1
i32.const 24
i32.ne
if
i32.const 0
return
end
```

3)Find the encrypted data

```
(data (i32.const 0)
  "\\15\\04\\1b\\06\\11\\18\\07\\08\\1c\\122<(,>++*(,0\\c4\\d
6\\d4"
)
```

This is **raw bytes**, not text. And you need to extract the bytes carefully!

4)Understand the XOR logic

```
key = 0x42 + (i * 3)
if (input[i] ^ key != enc[i]) fail
```

```
enc[i] = input[i] XOR key

input[i] = enc[i] XOR key
```

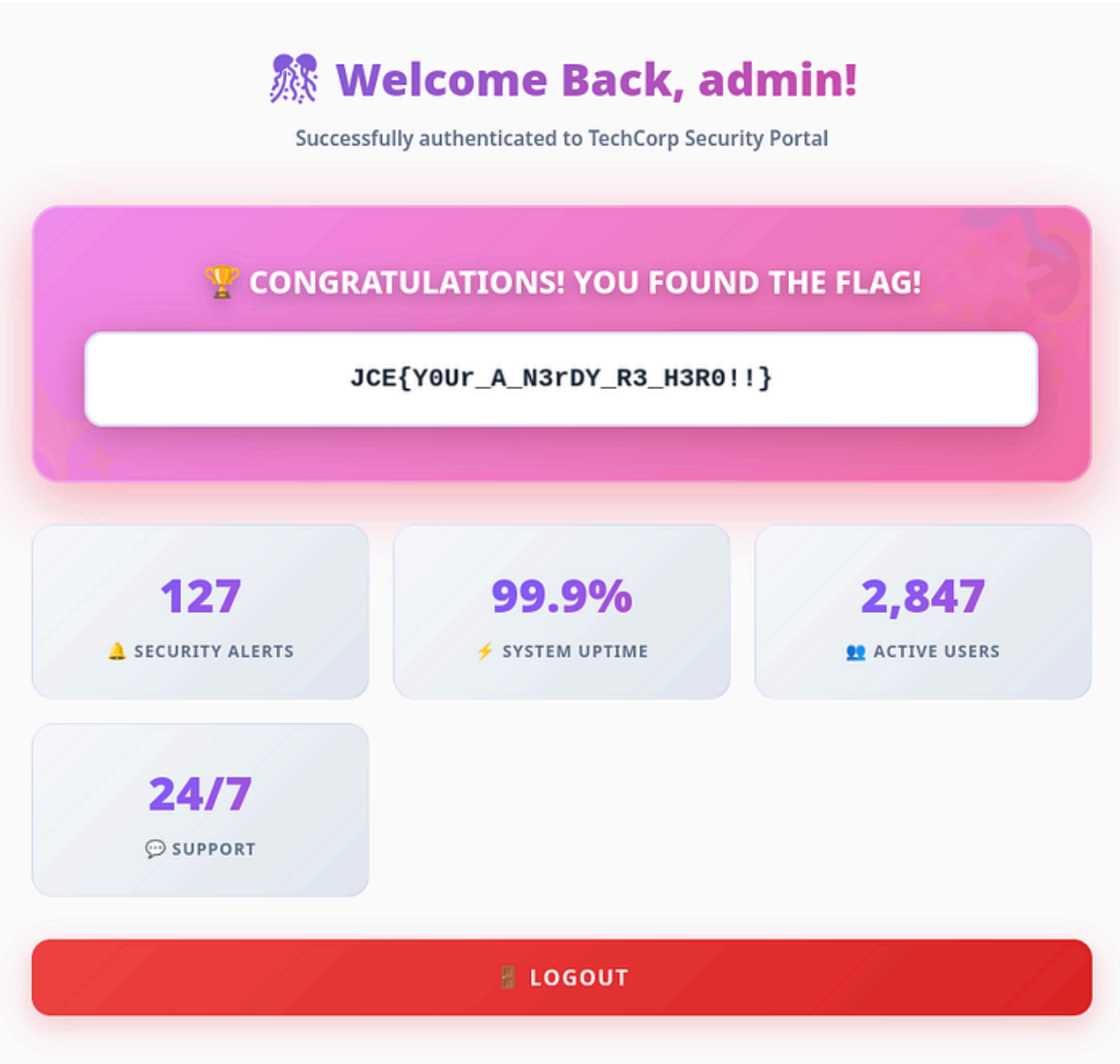
This is **fully reversible**.

5)Reconstruct the encrypted byte array

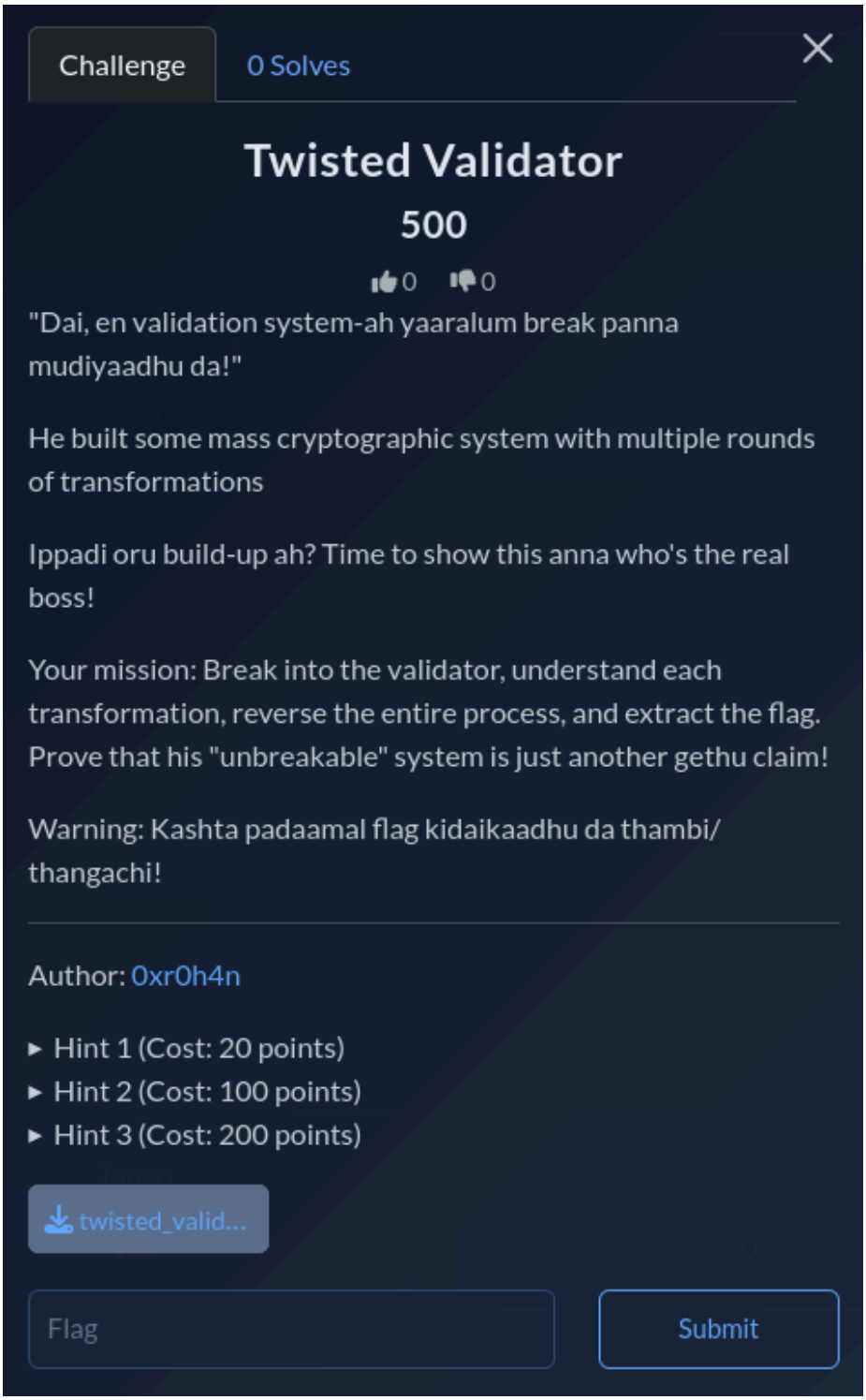
```
enc = [
  0x15,0x04,0x1b,0x06,0x11,0x18,0x07,0x08,
  0x1c,0x12,0x32,0x3c,0x28,0x2c,0x3e,0x2b,
  0x2b,0x2a,0x28,0x2c,0x30,0xc4,0xd6,0xd4
]
out = ""
for i, b in enumerate(enc):
    out += chr(b ^ (0x42 + i*3))
print(out)


// OUTPUT: WASM_IS_FOR_NERDY_PWNERS
```

So, this is our password for “admin”. Let’s go get the admin to get the flag!

Press enter or click to view image in full size




```
Flag: JCE{Y0Ur_A_N3rDY_R3_H3R0!!}
```



File: twisted_validator.py

Ok, this challenge is all about Linear Congruential Generator, XORs, reversible operations and byte manipulation.

To know more about Linear Congruential Generator (LCG)

Initial Analysis:

The FlagChecker class has several methods:

`_mix()`: XOR operation with a key

`_rotate()`: Rotates bytes by n positions

`_scramble()`: Swaps adjacent bytes

`_generate_key()`: Linear Congruential Generator (LCG) for key generation

`process()`: Applies transformations in sequence

`verify()`: Checks if processed flag matches expected output

To find the flag, we need to **reverse** each operation:

Understanding the Expected Output (This is 26 bytes — matches our flag length)

```
expected = bytes([
    0x9e, 0x1c, 0x77, 0xd4, 0x8e, 0x97, 0x17, 0x58,
    0x73, 0x5e, 0xbe, 0x65, 0x44, 0xaf, 0x9c, 0x23,
    0x7d, 0x19, 0x6e, 0x89, 0xf2, 0x4d, 0x37, 0xc1,
    0x52, 0xa8
])
```

Reverse the Process. We need to apply operations in **reverse order!**

```
def reverse_process(data):

    key2 = generate_key(len(data), 0x1337 ^ 0xDEAD)
    data = mix(data, key2[::-1])

    data = unscramble(data)
    data = rotate_right(data, 7)

    key1 = generate_key(len(data), 0x1337)
    data = mix(data, key1)
```

```
    return data
```

Here’s the complete solution script:

```
def generate_key(length, seed):
    key = []
    current = seed
    for _ in range(length):
        current = (current * 1103515245 + 12345) & 0x7fffffff
        key.append(current & 0xff)
    return bytes(key)

def mix(data, key):
    output = []
    for i in range(len(data)):
        output.append(data[i] ^ key[i % len(key)])
    return bytes(output)

def rotate_right(data, n):
    n = n % len(data)
    return data[-n:] + data[:-n]

def unscramble(data):
    result = bytearray(data)
    for i in range(0, len(result) - 1, 2):
        result[i], result[i+1] = result[i+1], result[i]
    return bytes(result)

expected = bytes([
    0x9e, 0x1c, 0x77, 0xd4, 0x8e, 0x97, 0x17, 0x58,
    0x73, 0x5e, 0xbe, 0x65, 0x44, 0xaf, 0x9c, 0x23,
    0x7d, 0x19, 0x6e, 0x89, 0xf2, 0x4d, 0x37, 0xc1,
    0x52, 0xa8
])
```

```
key2 = generate_key(26, 0x1337 ^ 0xDEAD)
data = mix(expected, key2[::-1])

data = unscramble(data)
data = rotate_right(data, 7)

key1 = generate_key(26, 0x1337)
data = mix(data, key1)

try:
    flag = data.decode('latin-1') # Use latin-1 instead of ut
f-8 (important)
    print(f"Flag: {flag}")
except:
    print("Raw bytes:", data)

// OUTPUT: JCE{LCG_4nd_X0R_m4g1c!}
```

```
Flag: JCE{LCG_4nd_X0R_m4g1c!}
```